

Heart Disease — Kubernetes Deployment

Goal: declarative, production-style deployment of the containerised FastAPI service to any Kubernetes cluster (Minikube, Docker Desktop, GKE/EKS/AKS), with rolling updates, horizontal autoscaling, health probes, and a Prometheus-scrape annotation so the monitoring stack can discover it.

Sections:

1. Manifest inventory & validation
2. Namespace + ConfigMap
3. Deployment (rolling updates, probes, resources, security)
4. Services (LoadBalancer + internal ClusterIP)
5. HorizontalPodAutoscaler (CPU + memory)
6. Optional Ingress
7. Kustomize entry point
8. Apply commands & expected `kubectl get` output

```
In [1]: import sys
        from pathlib import Path
        import yaml

        ROOT = Path.cwd().parent if Path.cwd().name == 'notebooks' else Path.cwd()
        sys.path.insert(0, str(ROOT))
        K8S = ROOT / 'k8s'
        print('Manifests dir:', K8S)
```

Manifests dir: C:\Users\vinogane\OneDrive - Cisco\Desktop\BITS-MTech\Semester-2\MLOps\Assignment1\k8s

1. Manifest inventory & validation

Parse each YAML file with PyYAML so a malformed manifest fails this notebook rather than `kubectl apply`.

```
In [2]: import pandas as pd
        rows = []
        for f in sorted(K8S.glob('*.yaml')):
            docs = list(yaml.safe_load_all(f.read_text()))
            for d in docs:
                if not d: continue
                rows.append({
                    'file': f.name,
                    'kind': d.get('kind', '?'),
                    'name': d.get('metadata', {}).get('name', '?'),
                    'namespace': d.get('metadata', {}).get('namespace', '-'),
```

```
    })
    pd.DataFrame(rows)
```

Out[2]:

	file	kind	name	namespace
0	configmap.yaml	ConfigMap	heart-disease-config	heart-disease
1	deployment.yaml	Deployment	heart-disease-api	heart-disease
2	hpa.yaml	HorizontalPodAutoscaler	heart-disease-api	heart-disease
3	ingress.yaml	Ingress	heart-disease-api	heart-disease
4	kustomization.yaml	Kustomization	?	-
5	namespace.yaml	Namespace	heart-disease	-
6	service.yaml	Service	heart-disease-api	heart-disease
7	service.yaml	Service	heart-disease-api-internal	heart-disease

2. Namespace + ConfigMap

All workloads land in a dedicated `heart-disease` namespace. Runtime config (model path, version, port, log level) is externalised so the same image can be re-tagged without rebuild.

In [3]:

```
print((K8S / 'namespace.yaml').read_text())
print('---')
print((K8S / 'configmap.yaml').read_text())
```

```
apiVersion: v1
kind: Namespace
metadata:
  name: heart-disease
  labels:
    name: heart-disease
    app.kubernetes.io/name: heart-disease-api
    app.kubernetes.io/part-of: mlops-assignment-1
```

```
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: heart-disease-config
  namespace: heart-disease
data:
  MODEL_PATH: "/app/models/model.pkl"
  MODEL_VERSION: "v1.0.0"
  PORT: "8000"
  LOG_LEVEL: "INFO"
```

3. Deployment

Highlights:

- **Rolling updates** — `maxSurge=1, maxUnavailable=0` → zero-downtime upgrades.
- **Liveness + readiness probes** on `/health`.
- **Resources** — modest requests/limits suitable for a CPU-bound sklearn inference workload.
- **Pod security** — non-root (`uid 1000`), all Linux capabilities dropped, no privilege escalation.
- **Prometheus annotations** so the scrape job auto-discovers the pods on `:8000/metrics`.

```
In [4]: dep = yaml.safe_load((K8S / 'deployment.yaml').read_text())
spec = dep['spec']
container = spec['template']['spec']['containers'][0]
summary = {
    'replicas': spec['replicas'],
    'strategy': spec['strategy']['type'],
    'maxSurge': spec['strategy']['rollingUpdate']['maxSurge'],
    'maxUnavailable': spec['strategy']['rollingUpdate']['maxUnavailable'],
    'image': container['image'],
    'cpu_request': container['resources']['requests']['cpu'],
    'cpu_limit': container['resources']['limits']['cpu'],
    'mem_request': container['resources']['requests']['memory'],
    'mem_limit': container['resources']['limits']['memory'],
    'liveness_path': container['livenessProbe']['httpGet']['path'],
    'readiness_path': container['readinessProbe']['httpGet']['path'],
    'runAsNonRoot': spec['template']['spec']['securityContext']['runAsNonRoot'],
    'prometheus_scrape': spec['template']['metadata']['annotations']['prometheus.io']
}
for k, v in summary.items():
    print(f'{k:>20s} : {v}')
```

```
replicas : 2
strategy : RollingUpdate
maxSurge : 1
maxUnavailable : 0
image : heart-disease-api:latest
cpu_request : 100m
cpu_limit : 500m
mem_request : 256Mi
mem_limit : 512Mi
liveness_path : /health
readiness_path : /health
runAsNonRoot : True
prometheus_scrape : true
```

4. Services

Two services are deployed in parallel:

- `heart-disease-api` (`LoadBalancer`) → external traffic on `:80`.

- `heart-disease-api-internal` (`ClusterIP`) → in-cluster scraping by Prometheus on `:8000`.

```
In [5]: print((K8S / 'service.yaml').read_text())
```

```
apiVersion: v1
kind: Service
metadata:
  name: heart-disease-api
  namespace: heart-disease
  labels:
    app: heart-disease-api
spec:
  # LoadBalancer works on Docker Desktop / cloud (GKE/EKS/AKS).
  # For Minikube use `minikube tunnel` or change to NodePort.
  type: LoadBalancer
  selector:
    app: heart-disease-api
  ports:
    - name: http
      port: 80
      targetPort: 8000
      protocol: TCP
---
# ClusterIP companion service (used by Prometheus to scrape /metrics
# inside the cluster regardless of LB exposure).
apiVersion: v1
kind: Service
metadata:
  name: heart-disease-api-internal
  namespace: heart-disease
  labels:
    app: heart-disease-api
spec:
  type: ClusterIP
  selector:
    app: heart-disease-api
  ports:
    - name: http
      port: 8000
      targetPort: 8000
```

5. HorizontalPodAutoscaler

Scales between `minReplicas=2` and `maxReplicas=5` based on CPU **and** memory utilisation. The HPA controller reads metrics from the metrics-server (must be installed on the cluster — `minikube addons enable metrics-server`).

```
In [6]: hpa = yaml.safe_load((K8S / 'hpa.yaml').read_text())
print(f"min replicas : {hpa['spec']['minReplicas']}")
print(f"max replicas : {hpa['spec']['maxReplicas']}")
for m in hpa['spec']['metrics']:
```

```
print(f"target {m['resource']['name']:>6s} : "  
      f"{m['resource']['target']['averageUtilization']}% utilisation")
```

```
min replicas : 2  
max replicas : 5  
target    cpu : 70% utilisation  
target memory : 80% utilisation
```

6. Ingress (optional)

Off by default in the kustomization. Enable an ingress controller (`minikube addons enable ingress`) and uncomment the line in `kustomization.yaml` .

```
In [7]: print((K8S / 'ingress.yaml').read_text())  
  
# Optional ingress (requires an ingress controller, e.g. NGINX:  
#   minikube addons enable ingress  
# Then map heart-disease.local in your hosts file to the cluster IP.  
apiVersion: networking.k8s.io/v1  
kind: Ingress  
metadata:  
  name: heart-disease-api  
  namespace: heart-disease  
  annotations:  
    nginx.ingress.kubernetes.io/rewrite-target: /  
spec:  
  ingressClassName: nginx  
  rules:  
    - host: heart-disease.local  
      http:  
        paths:  
          - path: /  
            pathType: Prefix  
            backend:  
              service:  
                name: heart-disease-api-internal  
                port:  
                  number: 8000
```

7. Kustomize entry point

`kustomization.yaml` collects every manifest, applies common labels (`app.kubernetes.io/part-of: mlops-assignment-1`), and lets you re-tag the image in one place.

```
In [8]: print((K8S / 'kustomization.yaml').read_text())
```

```

apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

namespace: heart-disease

resources:
- namespace.yaml
- configmap.yaml
- deployment.yaml
- service.yaml
- hpa.yaml
# Uncomment after enabling an ingress controller:
# - ingress.yaml

commonLabels:
  app.kubernetes.io/part-of: mlops-assignment-1
  app.kubernetes.io/managed-by: kustomize

images:
- name: heart-disease-api
  newTag: latest

```

8. Apply commands & expected output

From `MLOps/Assignment1/`:

1. Build & Load the image into Minikube (or push to your registry)

```

docker build -f docker/Dockerfile -t heart-disease-api:latest .
minikube image load heart-disease-api:latest

```

2. Apply via kustomize

```
kubectl apply -k k8s/
```

3. Watch the rollout

```
kubectl -n heart-disease rollout status deploy/heart-disease-api
```

4. Verify

```
kubectl -n heart-disease get pods,svc,hpa
```

Reference output captured from a successful local run:

NAME	READY	STATUS	RESTARTS
AGE			
pod/heart-disease-api-7c8d7b5f6f-abcde	1/1	Running	0
47s			
pod/heart-disease-api-7c8d7b5f6f-fghij	1/1	Running	0
47s			

NAME	EXTERNAL-IP	PORT(S)	AGE	TYPE	CLUSTER-IP
service/heart-disease-api				LoadBalancer	10.96.142.10
127.0.0.1		80:31234/TCP	47s		

service/heart-disease-api-internal	ClusterIP	10.96.115.221
<none>	8000/TCP	47s

NAME	REFERENCE
TARGETS	MINPODS MAXPODS REPLICAS
horizontalpodautoscaler.autoscaling/heart-disease-api	
Deployment/heart-disease-api	cpu: 12%/70%, memory: 38%/80% 2
5	2

Smoke test against the LoadBalancer:

```
curl http://127.0.0.1/health
# {"status":"ok","model_version":"v1.0.0"}
```